

SKILL_WEEK-10

Task 1: To Retrieve the current directory, display the path, process exit requests, clean up resources, save state, and verify correct behavior.

```
UW PICO 5.09
    break;
}

if (strcmp(command, "exit") == 0)
{
    printf("\nExit request received.\n");
    state_file = fopen("skill10_state.txt", "w");
    if (state_file != NULL)
    {
        fprintf(state_file,
            "Last Working Directory: %s\n",
            cwd);

        fclose(state_file);

        printf("Program state saved successfully.\n");
    }
    else
    {
        printf("Unable to save state.\n");
    }

    printf("Cleaning up resources...\n");
    printf("Program terminated successfully.\n");
    break;
}
else
{
    printf("Unknown command: %s\n", command);
}

return 0;
}
```

Fig 1: C program for retrieving the current directory, handling exit requests, saving state, and performing cleanup

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/skill10_task1.c -o bin/skill10_task1
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task1
Current Working Directory:
/Users/amrutha/OSSP

Enter command (type 'exit' to quit): hello
Unknown command: hello
```

Fig 2: Handling an unknown command while the program remains active

```
Enter command (type 'exit' to quit): exit

Exit request received.
Program state saved successfully.
Cleaning up resources...
Program terminated successfully.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 3: Processing the exit request, saving program state, and cleaning up resources

```

amrutha@Amruthas-MacBook-Air OSSP % cat skill10_state.txt
Last Working Directory: /Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %

```

Fig 4: Verification of the saved program state after graceful termination

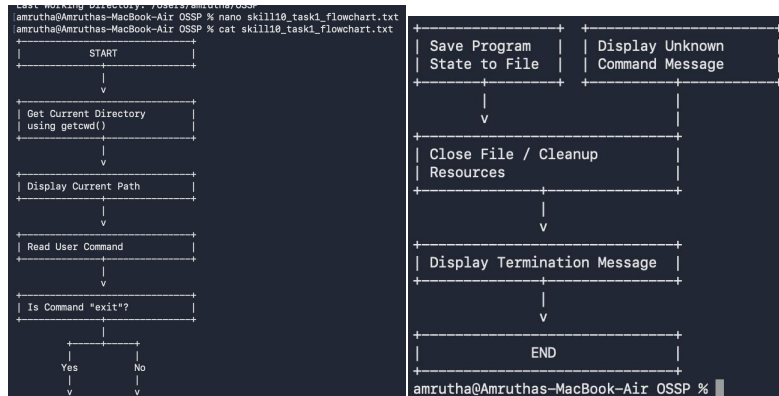


Fig 5: Flowchart for retrieving the current directory, saving state, and handling graceful program termination

Result:

The current working directory was retrieved and displayed successfully. Exit requests were processed correctly, the program state was saved to a file, resources were cleaned up, and the program terminated gracefully.

Task 2: Parse export syntax, update environment variables, validate names, handle existing variables, export to child processes, and test functionality.

```

LW PICO 5.09
}
return 1;

if (setenv(name, value, 1) != 0)
{
    perror("setenv failed");
    return 1;
}

printf("Environment variable updated successfully.\n");
printf("%s = %s\n", name, getenv(name));

pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

if (pid == 0)
{
    printf("\nChild Process:\n");
    printf("Inherited %s = %s\n", name, getenv(name));
    exit(0);
}
else
{
    wait(NULL);

    printf("\nParent Process:\n");
    printf("Export verification completed.\n");
}

return 0;
}

```

Fig 6: C program for parsing export syntax and updating environment variables

Test Case 1:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task2
Enter export command: export COURSE=OSSP
Environment variable updated successfully.
COURSE = OSSP

Child Process:
Inherited COURSE = OSSP

Parent Process:
Export verification completed.
```

Fig 7: Successful export of an environment variable and inheritance by the child process

Test Case 2:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task2
Enter export command: export COURSE=OperatingSystems
Environment variable updated successfully.
COURSE = OperatingSystems

Child Process:
Inherited COURSE = OperatingSystems

Parent Process:
Export verification completed.
```

Fig 8: Updating an existing environment variable using export syntax

Test Case 3:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task2
Enter export command: export 1COURSE=OSSP
Invalid variable name: 1COURSE
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 9: Detection of an invalid environment variable name

Test Case 4:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/skill10_task2
Enter export command: export COURSE
Invalid syntax. '=' is missing.
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 10: Handling invalid export syntax with a missing assignment operator

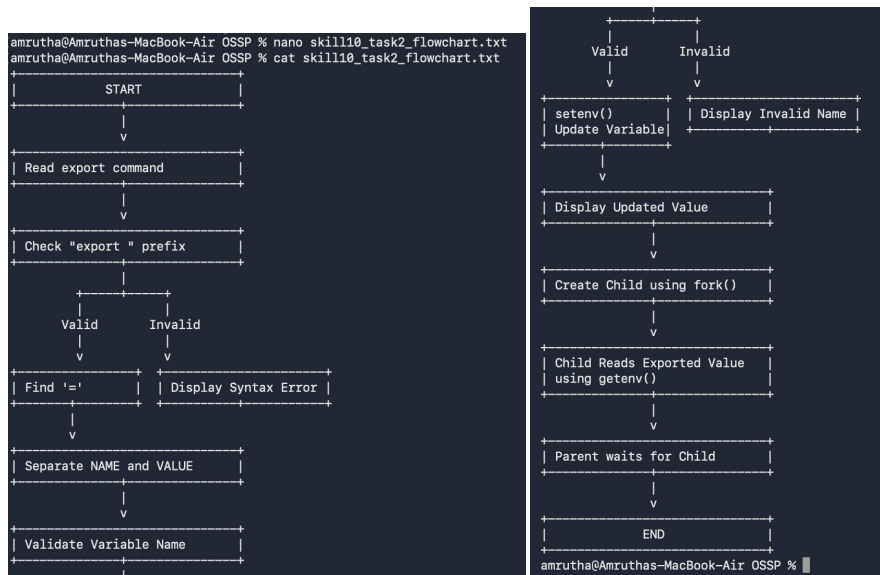


Fig 11: Flowchart for parsing export syntax, updating environment variables, and verifying child-process inheritance

Result:

Export syntax was parsed successfully, valid environment variable names were accepted, existing variables were updated using `setenv()`, invalid inputs were handled correctly, and exported variables were inherited by the child process.